Oracle Observability of Quantum Transpiler Regressions

What Output-Equivalence Oracles Miss in Qiskit, and a Leakage-Safe, Cost-Aware Regression-Testing Pilot

Furqan Nasir[1,2], Arif Shah[1], Iftikhar Alam[1]

[1] City University of Science and Information Technology (CUSIT), Peshawar, Pakistan (PhD Student, Computer Science)

[2] Department of Computer Science, National University of Computer and Emerging Sciences (FAST-NUCES), Islamabad, Pakistan (Lecturer)

Corresponding author: Furqan Nasir (furqannr@gmail.com) · ORCID: Furqan Nasir 0000-0002-5259-3448, Arif Shah 0000-0003-0090-3333

Abstract

Quantum compilers such as Qiskit's transpiler change frequently, and a pass modification can introduce a correctness, circuit-quality, or compilation-performance regression. Our central finding is an oracle-observability gap: across 26 merged Qiskit transpiler bug-fixes, about 38% (Wilson 95% CI [22%, 57%]) lie in channels invisible to black-box output-equivalence oracles, namely corrupted layout/contract metadata, non-determinism, and dropped global phase, anchored by a built-from-source case (ElidePermutations, H1) that an output oracle cannot distinguish from its fix yet a fault-class-matched isolated-pass/property oracle detects. Output equivalence is thus an incomplete correctness criterion for the transpiler. To study this rigorously we contribute a leakage-safe evaluation methodology and open prototype, cart: cohort-separated change events, layout-aware oracles with an empirical false-positive rate (0.068), provenance control, six implementation-validity gates, a strict claim-scope rule, and a fault-manifestation taxonomy. Piloted on Qiskit 2.x, the pipeline passes all six gates and 64 automated tests, and we verify one real forward-regression event from source. We additionally report cost-aware regression-test selection as an explicit, documented null: at this micro-corpus scale (a 1.65 CPU-second suite without cost pressure) no selector beats the simple baselines or random ordering, which an ablation attributes to its severity term. We frame the work as a reproducible feasibility study with a pre-specified path to a powered evaluation; all code and artifacts are released.

Keywords: Quantum software engineering · Quantum transpiler · Regression test selection · Quantum circuit equivalence checking · Qiskit · Test oracle observability

1. Introduction

Quantum software development kits (SDKs) such as Qiskit are developed at a rapid and continuous pace, and the compilers inside them, the transpilers that map an abstract circuit onto a hardware-native gate set and a limited qubit connectivity, are revised very often. A single change in a layout, routing, translation, optimization, or scheduling pass can introduce a regression of one of three kinds. The first is a correctness regression, in which the program crashes or the compiled circuit computes a different unitary. The second is a circuit-quality regression, in which the output carries more two-qubit gates or a larger depth and therefore loses fidelity on noisy hardware. The third is a compilation-performance regression, in which the transpilation itself becomes slower or consumes more memory. Empirical studies have already shown that compiler and transpiler components are an important source of defects in quantum platforms [1], and several recent Qiskit transpiler fixes fall into exactly these three categories.

Detecting such regressions early is, in essence, a regression-testing problem; the real difficulty is that the test space is very large. The atomic unit of testing is a tuple of the form (circuit, backend, transpiler configuration, oracle), and a realistic suite ranges over many circuit families and several device topologies. Existing benchmark suites, for instance, already enumerate on the order of a thousand

workloads for a single SDK [2]. Re-executing the complete suite together with its equivalence oracles on every change therefore exhausts any practical CI budget rather quickly.

Motivating scenario. Consider a developer who opens a pull request that modifies a routing heuristic. Most of the suite has nothing to do with this particular change, and yet the tight budget of a pre-merge pipeline permits only a small fraction of the tests to run. The question we ask is therefore the following: which tests, and in which order, give the best chance of revealing a regression introduced by this change, given that the cost of each test, transpilation together with oracle evaluation, varies a great deal from one test to another?

For conventional software this question is addressed by classical regression test prioritization, and decades of empirical evidence indicate that simple cost-, history-, and change-aware heuristics are strong baselines, frequently remaining competitive with much heavier learned models when labelled failure data are scarce [3, 4]. To the best of our knowledge, however, no earlier work has formalized or evaluated cost-aware regression test selection for the quantum transpiler in particular, where the faults are compiler regressions and the oracle must reason about circuit equivalence under a permutation of the qubit layout. Most existing quantum-testing research instead aims at finding bugs and generating tests through metamorphic or differential testing [5, 6, 7, 8], or at benchmarking SDKs [2]; this is complementary to our problem but distinct from the budgeted selection of existing tests for an incoming change.

This paper contributes, in order of strength, an empirical measurement of an oracle-observability gap, a leakage-safe evaluation methodology and open harness, and an honest feasibility pilot with a documented null. Concretely:

1. **An empirical measurement of oracle observability.** We quantify how often real Qiskit transpiler regressions escape black-box output-equivalence oracles: a repository-mining study of 26 merged transpiler bug-fixes finds about 38% (Wilson 95% CI [22%, 57%]) in output-invisible channels (corrupted layout/contract metadata, non-determinism, dropped global phase), and a built-from-source case (H1, ElidePermutations) that the output oracle cannot see but a fault-class-matched isolated-pass/property oracle can. The contribution is the measurement and the demonstration, since the observation itself follows from transpiler internals.
2. **A leakage-safe evaluation methodology and open harness.** We separate change events into non-pooled cohorts (forward regression, fix-boundary differential, mutation), apply layout-aware oracles with an empirical false-positive rate, calibrate cost with CPU process time, freeze the change–stage map, and enforce six implementation-validity gates with a strict claim-scope rule, together with a fault-manifestation taxonomy that classifies each regression by its detection channel.
3. **A prototype (cart) and an honest feasibility pilot, with a documented null.** The methodology is realized end to end (Section 11) and piloted on Qiskit 2.x. We report the cost-aware selection study as an explicit null baseline, not a positive result: at this micro-corpus scale (a ~1.65 CPU-second suite with no cost pressure) no selector, including the proposed risk_score, beats the simple baselines or random ordering, and an ablation traces the deficit to its severity term. Selection is a feasibility baseline for the pre-specified scale-up, not a claimed method.

We answer three research questions: RQ1 (is the leakage-safe pipeline operationally sound under an executable gate audit?), RQ2 (how does the transparent selector compare with the baselines (including random ordering) at this scale, and what can be concluded?), and RQ3 (which real historical transpiler regressions are observable through black-box oracles on commodity hardware, and which are merely unreproduced?). All comparative claims are scoped as pilot evidence. Section 2 gives background; Section 3 surveys related work; Section 4 presents the methodology; Section 5 the implementation; Section 6 the pilot; Sections 7–9 discuss findings, threats, and future work; Section 10 concludes; Section 11 is the artifact statement.

2. Background

2.1 The Qiskit transpiler

Qiskit compiles an abstract circuit to a target backend through a staged pass manager [9]. The canonical stages are layout, routing, translation, optimization, and scheduling, bundled at optimization levels 0–3. Two properties matter for testing: the actually executed passes depend on the circuit, backend, configuration, and the exact Qiskit revision (so they must be observed, not inferred from source); and transpilation may permute qubits, so a transpiled circuit equals the original only modulo a layout permutation.

2.2 Transpiler regression types and their manifestation

We distinguish what kind of defect a regression is from how, and indeed whether, it shows up in an observable signal, because the gap between these two is the central finding of the pilot. The fault-type grouping itself is conventional. The correctness group contains functional faults (a compilation failure) and semantic faults (a different unitary or output). The quality group covers circuit-quality faults, such as a worse two-qubit count or a larger depth. The performance group covers compilation-performance faults. These groupings are not mechanical: a crash is not automatically a semantic fault, and an increase in two-qubit count is not a quality regression if the depth improves substantially. For that reason we judge quality with a Pareto rule over pre-declared thresholds and report the two metrics separately.

Crucially, two regressions of the same fault type can manifest in very different observable signals, and some manifest in signals a black-box, output-only oracle never inspects. We therefore use a fault-manifestation taxonomy, the channel through which a regression could be detected:

Table 1. Study-specific fault-manifestation taxonomy (detection channel).

Manifestation channel	Observable signal	Output oracle sees it?
output_semantic	different unitary/output state	Yes (within oracle width)
compilation_failure	crash, exception, invalid circuit	Yes
circuit_quality	worse two-qubit count or depth	Yes
performance	slower / more memory-hungry compilation	Yes, if over the screen
transpiler_contract_or_metadata	wrong internal property (TranspileLayout, final_layout, routing_permutation) with applied output still correct	No, invisible to output oracles
determinism_or_reproducibility	output varies across fixed-seed runs, often only under a specific (e.g. noisy) target	Only with a determinism oracle on that target

This taxonomy is what makes the historical results in Section 6.4 interpretable, since a regression can be entirely real and yet sit in a channel that our black-box pipeline never observes. The ElidePermutations regression (H1), for instance, manifests in the transpiler_contract_or_metadata channel: it corrupts a layout property rather than producing a wrong output unitary. The VF2Layout regression (H3) manifests as determinism_or_reproducibility under a noisy target, which is neither a circuit_quality nor an output_semantic signal. In both cases a “pass” from the output oracle is exactly what one should expect, and it is not evidence that the oracle under-detects quality or semantic faults.

2.3 The oracle problem for transpilation

Deciding whether a transpiled circuit is correct is an instance of the quantum oracle problem [10], which is the central obstacle catalogued in the general test-oracle survey [11]. When a circuit is small and unitary-pure, one can normalize the transpiled layout and then compare unitaries modulo global phase. This comparison is exact, but it does not scale, because an n-qubit operator already has 2^n × 2^n complex entries. Larger or non-unitary circuits must instead be checked with sampled-statevector, observable, metamorphic, or structural methods [5]. One consequence is central to our findings: a black-box oracle only observes the output, so a fault that corrupts internal metadata while leaving the output unitary intact can remain invisible to it. Dedicated quantum equivalence-checking methods exploit reversibility and simulation to compare a circuit before and after compilation efficiently [12], yet they too reason about the output map and not about a transpiler's internal contracts. This is exactly where our finding bites: that literature optimizes the efficiency of an output-map comparison but takes output equivalence as the correctness criterion, so it cannot, by construction, observe the contract/metadata-channel faults we quantify in §6.6.

2.4 Rate-of-detection metrics

Classically, prioritization is measured by APFD and by its cost-cognizant variant APFDc, which weights detection by cost and severity [13, 14]. APFDc, however, assumes that a single common ordering exposes multiple faults. In our setting each event typically has only one fault, ranked under its own ordering, so APFDc is not appropriate as a headline metric. We therefore rely on per-event detection-curve summaries, averaged across events, with the cohorts reported separately.

3. Related Work

We position the work across three threads and close with an explicit comparison.

3.1 Regression test selection and prioritization

Rothermel et al. introduced systematic test-case prioritization and the rate-of-fault-detection measure [15], and Elbaum et al. established APFD together with a family of empirical studies [13]. The cost-cognizant variant APFDc later incorporated varying test costs and fault severities [14], and it is the conceptual ancestor of our objective. In their survey, Yoo and Harman observe that change-, history-, and cost-aware heuristics remain persistently strong baselines [3]. The continuous-integration setting reinforces this. Lightweight selection and prioritization that avoid coverage instrumentation are markedly more cost-effective [16]. A large empirical study of readily available CI and version-control metadata finds that simple, well-known heuristics frequently outperform complex machine-learned models [17], and information-retrieval-based prioritization reports the same pattern, with well-tuned heuristics staying competitive [18]. This accumulated evidence is what leads us to a transparent selector and to a cautious reading of the pilot. Two adaptations are nevertheless required for transpilation. First, the fault is a compiler regression that spans correctness, quality, and performance, rather than a test failure in the program under test. Second, the oracle must decide circuit equivalence modulo qubit-layout permutation. We keep the cost-aware objective and the strong baselines, but we redefine the fault model, the oracles, and the metrics.

3.2 Learning-based test selection

At industrial scale, predictive (learned) test selection trains gradient-boosted models on large histories of test outcomes, and it can halve testing cost while keeping most of the failure signal [4]. Such methods are data-hungry, since they presuppose a large labelled corpus of per-test outcomes, and no such corpus yet exists for quantum-transpiler regressions. A broad literature applies reinforcement learning [19], multi-armed bandits for volatile test pools [20], and other learners surveyed systematically [21] to CI test prioritization, and all of them require substantial historical signal, which only reinforces our staged plan. Because simple heuristics are known to stay competitive when labelled data are sparse [3], we propose a transparent selector first and reserve a learned comparison for a later scale-up under identical anti-leakage rules.

3.3 Quantum software and compiler testing

Quantum software testing must also contend with the oracle problem [10]. Metamorphic and differential testing are two ways to mitigate it. MorphQ, for example, generates diverse programs and applies quantum-specific metamorphic relations to test Qiskit, and in doing so exposes real bugs [5], while MorphQ++ reproduces and extends metamorphic testing of quantum compilers [6]. Quantum software engineering is by now an established research thread, including within this venue [22], where the transpiler itself is an active optimization target [23]. Several recent frameworks sharpen the picture. QuCheck offers property-based testing of quantum programs in Qiskit, with flexible generators, assertions, and preconditions that are evaluated through mutation analysis [7]. QEMI adapts equivalence-modulo-inputs compiler testing to quantum stacks, generating program variants by removing dead code and uncovering crash and behavioural bugs across Qiskit, Q#, and Cirq [8]. QDiff applies differential testing to quantum software stacks, exploring semantics-preserving variants and filtering circuits by static characteristics to compare Qiskit, Cirq, and Pyquil [24], and QSPE enumerates skeletal program variants with statevector-based validation, reporting, in a preprint, 81 Qiskit miscompilations that developers have acknowledged [25]. Concolic testing targets transpiler decision points, parameter thresholds, and layout heuristics, surfacing missed optimizations and semantic deviations in Qiskit passes [26]. A complementary, formal line of work verifies the compiler directly: Giallar provides push-button verification of Qiskit passes, verifying 44 of 56 passes across 13 versions and uncovering three bugs [27]. Static analysis and bug benchmarks round out the landscape, with LintQ detecting quantum-specific defects that general linters miss [28] and Bugs4Q curating real, reproducible Qiskit bugs [29]. Mutation analysis has likewise been adapted to quantum programs in Muskit [30] and QMutPy [31], and a recent roadmap explicitly names transpilation as an under-studied testing target [32]. Empirical studies confirm that compiler and transpiler components are a significant source of defects [1], and Benchpress benchmarks circuit construction, manipulation, and compilation across SDKs [2]. All of these efforts aim at bug finding, test generation, or benchmarking; much like MorphQ, they create or transform programs in order to expose defects, rather than select among an existing budgeted suite. None of them addresses budgeted selection of existing tests for an incoming transpiler change with quantum-aware oracles, cohort-separated events, CPU-time cost calibration, and leakage-safe temporal evaluation. To our knowledge, cost-aware regression test selection for the quantum transpiler has not previously been formalized or piloted.

3.4 Positioning

Table 2 contrasts the closest threads along the axes that matter for budgeted CI of a compiler. Within this very collection, Open QBench benchmarks the performance of quantum-computing platforms [33]; our focus is complementary, in that we select and order existing tests for an incoming transpiler change rather than benchmark platforms. The distinguishing combination of our work is the rightmost five columns taken together: a quantum-aware oracle applied to selecting and ordering existing tests under an explicit cost budget, with leakage-safe temporal evaluation and cohort separation. Individually each property appears somewhere in prior work; in combination, for the quantum transpiler, it does not. (● = yes, ◐ = partial, ○ = no/not applicable.)

Table 2. Positioning relative to prior work.

Approach	Primary goal	QA oracle	Cost-aware	Selects/orders existing	Leakage-safe temporal	Cohort sep.
Classical RTP/TCP	prioritize tests of a program	○	●	●	◐	○
Predictive selection	learned selection at scale	○	●	●	●	○
MorphQ / MorphQ++	metamorphic bug finding	●	○	○	○	○

QuCheck	property-based testing	●	○	○	○	○
QEMI	EMI stack bug finding	●	○	○	○	○
QDiff	differential stack bug finding	●	○	○	○	○
QSPE	skeletal-program differential testing	●	○	○	○	○
Giallar	formal Qiskit-pass verification	●	○	○	○	○
Benchpress	SDK compilation benchmarking	◐	○	○	○	○
This work	cost-aware test SELECTION for the transpiler	●	●	●	●	●

4. Methodology

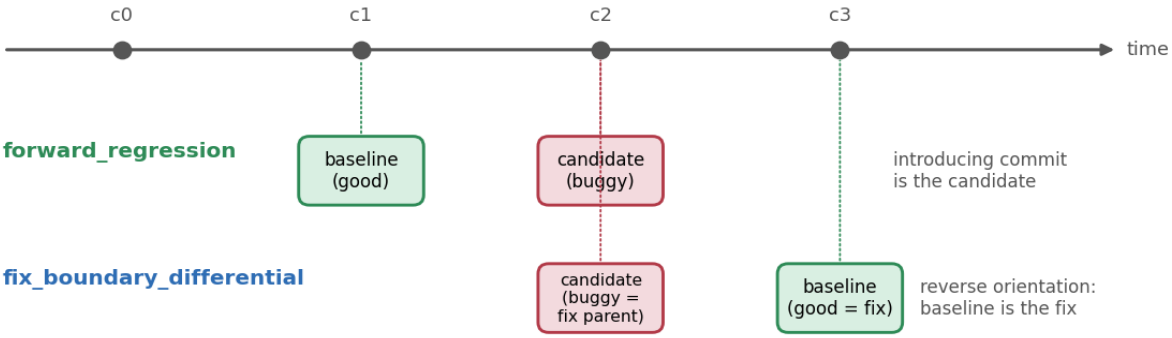
4.1 Problem formulation

Given a transpiler change and a fixed per-event budget B, select and order tests to maximize early detection of regressions. Conceptually each test is scored by expected_value ≈ P(regression | change, circuit, backend) × impact ÷ expected_cost, and tests are chosen greedily within B. P(·) is the conceptual target; the proposed selector realizes it with a transparent surrogate (§4.5). The primary evaluation unit is the (event_id, test_id) pair.

4.2 Change-event model and cohorts

The unit of the dataset is a change event, not a commit: an event is a tuple (baseline_revision, candidate_revision, change_metadata, fault_id), each carrying a unique fault_id and a fault_type. Every event declares one of three cohorts, and the cohorts are reported separately and never pooled. A forward_regression event pairs the last known-good parent (the baseline) with the regression-inducing commit (the candidate), and it is the only cohort that models the real CI question. A fix_boundary_differential event is used in reverse orientation when the introducing commit has not been traced: it pairs a fix commit (the baseline) with its parent (the candidate), which yields a fault-revealing differential that we never describe as a prediction of an incoming change. A mutation event injects a single controlled fault on a base revision and is used only to fill gaps in fault-type coverage. Throughout, historical events are preferred. Figure 2 illustrates the three cohorts and their baseline/candidate orientation.

Figure 2. Cohort orientations on a commit timeline (reported separately).



4.3 Test units, manifest, and provenance

A test unit is a tuple (circuit, backend, transpiler_configuration, oracle). A static manifest stores its metadata. A separate calibrated profile records the measured baseline cost, the pass stages that actually executed (obtained through instrumentation rather than inferred from filenames), routing observations, and peak memory; it is keyed by (event_id, baseline_sha, test_id, event_environment_id). Each unit also declares its provenance, one of pre_existing, extracted_from_fix, or synthesized, together with an available_before_candidate flag, and these support the two evaluations described in §4.8.

4.4 Oracles

Semantic. The semantic oracle is tiered by applicability and width. For a small circuit (at most 12 qubits) that is unitary-pure and ancilla-free, it checks exact unitary equivalence modulo global phase after first normalizing the layout. For every other circuit it falls back to structural validity, meaning adherence to the basis and to the coupling map. A full 2^n operator is therefore never materialized for large circuits. This mirrors dedicated quantum equivalence-checking practice, which compares the maps of the pre- and post-compilation circuits and exploits reversibility and simulation to stay tractable [12].

Quality. A Pareto rule over pre-declared thresholds; Δ two-qubit count and Δ depth reported separately; a sensitivity grid is reported and thresholds are fixed before held-out evaluation.

Performance. A two-stage protocol (median-slowdown screen, then robust effect size); exploratory on a laptop, timed with CPU process time.

Empirical false-positive rate. A warning is a false positive only if not upheld by independent confirmation, never merely because an oracle fired.

4.5 The proposed transparent selector (risk_score)

The selector introduces no ML and outputs an interpretable score, not a calibrated probability (Figure 3 shows the score's multiplicative structure):

```
risk_score = (ε + change_stage_match) × circuit_backend_sensitivity ×
impact_prior × oracle_confidence ÷ expected_cost × novelty_multiplier
```

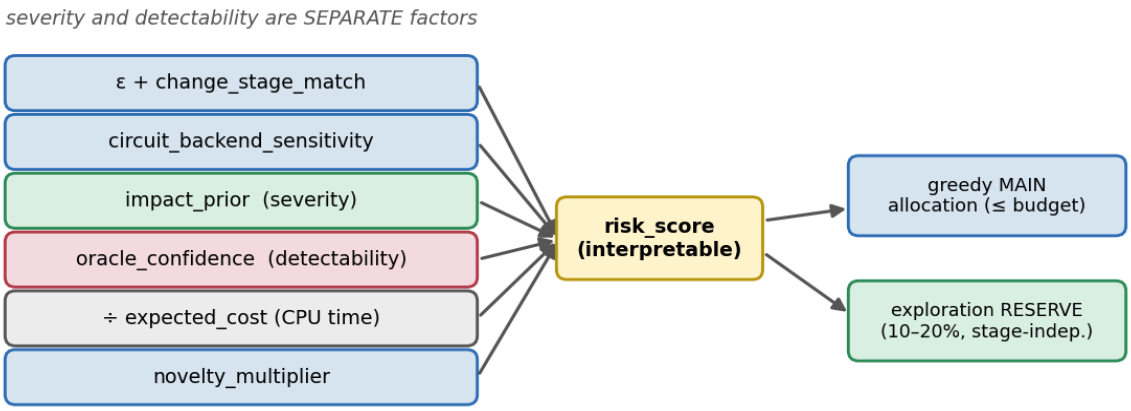


Figure 3. The transparent risk_score selector and budgeted selection.

Fault severity (impact_prior) and detection confidence (oracle_confidence) enter as separate multiplicands, so a high-impact test that relies on a weaker oracle is not penalized twice. The ε floor, together with a stage-independent exploration reserve of 10–20%, keeps an incomplete change–stage map from driving useful tests to zero, and unknown or framework-wide changes are routed into that reserve. When no prior qualifying events exist, a neutral cold-start fallback applies. The selector sees only pre-execution information: the candidate's fault_type, runtime, quality, and labels are never inputs. We stress that the criticality, oracle-confidence, and cost weights (Table 3) are hand-set rather than learned or calibrated; the selector is therefore a transparent, documented heuristic baseline, not a tuned method, and its components are assessed by ablation (§6.4) rather than presented as validated.

Table 3. Frozen risk_score configuration (pre-declared before evaluation).

Parameter	Value / rule
epsilon (change-stage floor)	0.05
exploration reserve	15% of budget (admissible range 10–20%)
novelty decay (per repeated family×backend)	0.5^count
circuit criticality (impact_prior)	GHZ 1.0, QFT 1.5, linear-QAOA 1.3, random-Clifford 1.2
oracle_confidence	unitary 1.0, sampled-SV 0.8, property 0.5, structural 0.3
oracle cost estimate (s)	unitary 0.05, sampled-SV 0.20, property 0.05, structural 0.01
circuit_backend_sensitivity	(1+pressure) × (1+two-qubit density) × (1 + n/30)
impact_prior	criticality × (1+pressure) × (1 + prior-failure-rate)
cold-start	prior-failure-rate = 0 ⇒ impact factor 1.0
tie-breaking / determinism	ties broken by test_id; seed-deterministic ranking

4.6 Baselines and budgeted selection

Five baselines observe the same budget: random, cheapest-first, diversity-only, history-only, and change-stage heuristic [3]. Selection reserves 10–20% of the budget for stage-independent exploration, fills the main allocation greedily by score, ensures ≥ 1 representative per circuit family where the budget permits, never exceeds the budget, and is seed-deterministic.

The baselines behave as follows. Random applies a seeded uniform shuffle of the candidate units. Cheapest-first orders them by ascending expected cost. Diversity-only greedily covers (circuit family × backend) combinations in ascending order of cost, and then adds the cheapest remaining units. History-only ranks units by descending prior failure rate, computed only from events strictly before the event date, breaks ties by cost, and reduces to cheapest-first under a cold start. Change-stage prefers units whose declared pass stages include the modified stage, again breaking ties by cost.

4.7 Metrics

Per event and averaged, cohorts/sources separated: Recall@budget, normalized TTFR, detection rate at {5,10,20,40}% budget, area under the detection-vs-budget curve, selection overhead, diversity coverage, and the empirical oracle FP rate. APFDc is reported only for the atypical multiple-fault-per-ordering case.

4.8 Anti-leakage controls

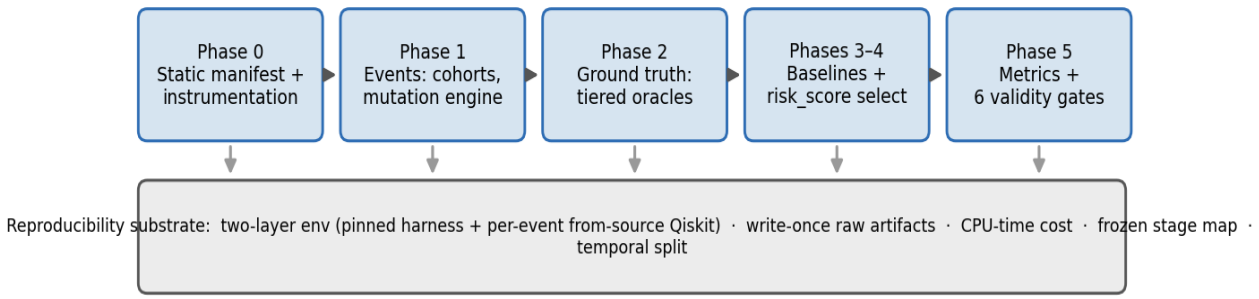
Four controls enforce leakage safety. The first is a group-level temporal split keyed by split_group_id = base_revision + mutation_family, which sends any group that straddles the cutoff entirely to the test side. The second is provenance-driven: a Primary CI evaluation restricts the candidates to tests that were available before the candidate, whereas post-fix targeted triggers are confined to a Secondary retrospective evaluation. The third freezes the change–stage map before held-out evaluation, and any later refinement is performed on training events only and is versioned. The fourth calibrates cost with CPU process time (time.process_time) and retains wall-clock time only as a secondary reference.

4.9 Validity gates and claim scope

Six gates check implementation correctness before any result is reported: budget compliance, reproducibility, absence of leakage, label reproducibility from the write-once artifacts, metric correctness against hand-computed cases, and oracle coverage. A claim-scope rule then forbids any headline temporal-generalization claim while fewer than three forward_regression events are verified in the held-out cohort, and in that situation results are reported per event, as pilot evidence. Whether the selector actually beats the baselines is treated as the empirical outcome, never as a precondition for reporting.

5. Implementation

Figure 1. The cart pipeline (Phases 0–5) over a shared reproducibility substrate.



The methodology is realized in a Python prototype, cart (cost-aware regression testing), organized by phase (manifest, events, oracles, labels, selectors, metrics, gates) with a CLI (manifest, events, ground-truth, select, evaluate, historical-run). Figure 1 summarizes the pipeline over its shared reproducibility substrate.

Two-layer environment. Reproducing historical regressions means building several different Qiskit revisions, which is incompatible with a single pinned Qiskit. We therefore separate a fixed harness

layer (Python 3.11, Benchpress, and the oracle and measurement stack, all lockfile-pinned) from a Qiskit-under-test layer that is built from source for each event. Instrumentation is anchored on Qiskit 2.4.2 within the 2.x window. Building from source requires a Rust toolchain.

Per-event from-source runner. For historical events, a runner builds the baseline and candidate revisions into isolated virtual environments. In each environment a worker runs as a subprocess, transpiles the test units with its own Qiskit, and serializes the outputs (as QPY) together with the CPU timings. The harness then loads both sets of results back and applies the shared oracle pipeline. In this way the system under test stays isolated, while a single oracle implementation is reused for both revisions.

Reproducibility artifacts. Raw oracle evidence is written once and never overwritten, and the derived labels are regenerated from it so that they reproduce exactly, including the timing-dependent fields. The per-event environment recipes, the seeds, the pre-declared thresholds, the frozen stage map, and the frozen per-run configurations are all committed. The prototype also ships an automated test suite, with the six validity gates wired directly into evaluate.

What is implemented versus exercised in this pilot. To avoid over-stating coverage we separate three states.

- **Implemented and exercised in the primary results:** the static manifest and instrumented profiling; the mutation engine (nine semantics-preserving quality operators); the tiered semantic oracle (exact ≤ 12 qubits, structural above); the quality Pareto oracle; the five baselines and the proposed risk_score selector; budgeted selection; the metric suite; the six validity gates; and the held-out evaluation on the nine-event × 32-unit (288-label) mutation cohort.
- **Implemented but not part of the primary results:** the per-event from-source historical runner (used to attempt H1/H3/H4, which are respectively rejected, blocked, and verified); the targeted-trigger units and the Secondary retrospective evaluation; the exploratory performance Stage-1 screen and the multi-run Stage-2 confirmation that verifies H4; a property/layout (contract/metadata) oracle wired into the runner that detects H1 on the Secondary retrospective track; and the incomplete_basis_translation functional-fallback operator.
- **Specified but deferred (not implemented here):** the sampled-statevector oracle tier (13–22 qubits); a noisy-target determinism oracle; three-run-median performance on controlled hardware; and any learned selector.

6. Evaluation

We evaluate the three research questions on a micro corpus, on a single 16 GB laptop, anchored on Qiskit 2.4.2.

6.1 Corpus and setup

The corpus has fourteen change events (Table 4): nine controlled quality mutations on the 2.4.2 base and five historical Qiskit transpiler regressions identified from fix pull requests, each with exact candidate/baseline commit SHAs. To stay within a single 16 GB laptop the test corpus is width-capped at 8 qubits, enumerating 4 circuit families × 2 widths × 2 backend topologies × 2 optimization levels = 32 test units (Table 5); every unit lies within the exact-unitary oracle tier. The Primary CI evaluation runs on the verified mutation cohort, nine operators × 32 units = 288 (event, test) labels, because the five historical events require per-event from-source builds and are reported separately in the audit (Table 8). All nine operators have intended_mutation_target = quality; eight of the nine are detected by the black-box oracle on this corpus (only drop_vf2_post_layout is not), and because a quality-targeted mutation can still surface other observable labels for particular circuit–backend configurations we distinguish intended_mutation_target from observed_test_unit_label. The primary-CI eligibility rule (Section 4.8) is implemented and exercised synthetically on this mutation cohort and is now additionally validated on one real forward-regression event (H4, §6.5), with fewer than three verified

so far. Thresholds, seeds, and the stage map are pre-declared/frozen before evaluation, and no selector parameter was tuned on any event. Controlled mutation is an established way to construct faulty quantum-circuit benchmarks with varied, characterizable detectability [34], and our reporting, covering objects under test, baselines, setup, configuration, and artifact support, follows recent guidance on empirical quantum-software-testing studies [35].

Table 4. Audited event ledger (cohorts never pooled).

Event(s)	Cohort	Fault type	Status
9 mutation operators	mutation	quality	verified (pilot)
H4 VF2PostLayout no-op (#14120)	forward_regression	performance	verified (Stage-2 multi-run)
H1 ElidePermutations (#14603)	fix_boundary	semantic	rejected (production-unobservable)
H3 VF2Layout determinism (#14730)	fix_boundary	quality	blocked (needs noisy target)
H2 final_layout composition (#14919)	fix_boundary	semantic	blocked (not run)
H5 VF2Layout panic (#16285)	fix_boundary	functional	blocked (not run)

Table 5. Test corpus composition (32 units; width-capped at 8 qubits to keep compute low).

Dimension	Values	Count
Circuit family	GHZ, QFT, linear-QAOA, random-Clifford	4
Width (qubits)	5, 8	2
Backend topology	linear, heavy-hex	2
Optimization level	1, 3	2
Total units		32
Semantic oracle tier	exact-unitary (all widths ≤ 12)	32 units

Table 6. Per-unit CPU-cost distribution across the 32 corpus units (mean cost per unit across the nine mutation events).

Statistic	Value (CPU seconds)
minimum	0.00008
median	0.0082
mean	0.0515
75th percentile	0.0343
maximum	0.3256
coefficient of variation	1.85
full-suite total	1.65

Because of the 8-qubit cap, the expensive 12-qubit line-topology units are excluded, so the suite stays cheap (at most 0.33 s per unit) while remaining moderately heterogeneous (CV ≈ 1.85). Broken down by width × backend, the mean per-unit cost ranges from 0.008 s for a 5-qubit unit to 0.17 s for an 8-qubit unit on the line topology. Cost variation is therefore present, but, as the results show, it is not the limiting factor. In absolute terms the entire 32-unit suite runs in roughly 1.65 CPU-seconds (median

0.0082 s per unit), so this pilot does not yet instantiate the cost-pressure regime that motivates cost-aware selection. That regime is exactly what the deferred scale-up targets, by adding expensive oracle tiers and events whose detecting tests are sparse or costly; until then, cheap detectors are abundant and almost any ordering succeeds early.

Table 7. Mutation ledger: verified cohort (9 operators × 32 units = 288 labels). Best-selector event AUC; 8 of 9 operators are detected by the black-box oracle.

Operator	Target stage	Fault type	Sem. kept	Detected?	Best AUC
disable_optimization_loop	optimization	quality	yes	yes	0.98
drop_vf2_post_layout	layout / opt	quality	yes	no	0.00
downgrade_routing	routing	quality	yes	yes	1.00
insert_redundant_cx_pair	optimization	quality	yes	yes	1.00
drop_optimization_stage	optimization	quality	yes	yes	1.00
force_opt_level_1	optimization	quality	yes	yes	1.00
force_opt_level_0	optimization	quality	yes	yes	1.00
downgrade_layout_trivial	layout	quality	yes	yes	1.00
insert_redundant_x_pair	optimization	quality	yes	yes	1.00

Table 8. Historical-event audit (5 events; none verified through black-box oracles in this pilot).

ID (PR)	Fault type → channel	candidate→baseline (short SHA)	Status	Reason
H1 (#14603)	semantic → contract/metadata	7c3890da→96fda188	rejected	built from source; corrupts virtual_permutation_layout, not the output unitary → production-unobservable
H2 (#14919)	semantic → contract/metadata	14df5941→dfcc5c6c	not run	expected to share H1's layout-property masking
H3 (#14730)	quality → determinism	056c6413→d33ef533	blocked	non-determinism tied to a noisy Target absent from our coupling-map backend
H4 (#14120)	performance → performance	a18e1516→a8d23667	verified	no-op overhead detected by a multi-run Stage-2 protocol over a width-extended symmetric regime (slowdown ≈1.9× at 16q to >300× at 27q, Cliff's δ=1.0); single-sample Stage-1 on the ≤8q corpus stayed below the 20% screen
H5 (#16285)	functional → compilation_failure	f0fae6f3→0b2cdf2b	not run	input-specific crash; not yet reproduced

Table 9. Controlled mutation operators and targeted regression triggers used in the pilot.

Name	Kind	Stage / target	Effect / oracle
disable_optimization_loop	mutation	optimization	remove peephole loop → residual redundancy (quality)
drop_vf2_post_layout	mutation	layout/opt	omit VF2PostLayout → worse layout (quality)
downgrade_routing	mutation	routing	force basic routing → more 2Q/depth (quality)
insert_redundant_cx_pair	mutation	optimization	append CX–CX identity → +2 2Q gates (quality)
drop_optimization_stage	mutation	optimization	disable the whole optimization stage (quality)
force_opt_level_1	mutation	optimization	downgrade optimization level to 1 (quality)
force_opt_level_0	mutation	optimization	downgrade optimization level to 0 (quality)
downgrade_layout_trivial	mutation	layout	force trivial layout method (quality)
insert_redundant_x_pair	mutation	optimization	append X–X identity → +2 1Q gates (quality)
incomplete_basis_translation	mutation (fallback)	translation	drop 1q basis → TranspilerError (functional)
trig-h1-elide-permutations	targeted (from fix)	ElidePermutations	PermutationGate+swaps; semantic oracle
trig-h2-final-layout-composition	targeted (from fix)	routing final_layout	identity-after-routing; semantic oracle
trig-h3-vf2-all-1q	targeted (from fix)	VF2Layout	all-1q; determinism (property) oracle

6.2 RQ1: Feasibility and rigor

The full pipeline runs from end to end. All six implementation-validity gates pass (Table 10), and the prototype passes 64 automated tests, among them the hand-computed metric checks and a leakage-invariance check. During validation we found and fixed two genuine defects: a CLI flag that silently dropped the targeted-run mode, and a semantic oracle that tried to materialize a 2^18 operator (about 512 GB) on an 18-qubit circuit, which is now capped and guarded by a regression test. We therefore answer RQ1 in the affirmative: the leakage-safe pipeline is operationally sound under the gate audit. The empirical oracle false-positive rate, computed over confirmed-versus-unconfirmed warnings on the mutation cohort, is 0.068.

Table 10. Implementation-validity gates.

Gate	Result
Budget compliance	PASS
Reproducibility (same seed ⇒ same ranking)	PASS
Leakage absence (current-event outcome cannot change ranking)	PASS
Label reproducibility from raw artifacts	PASS
Metric correctness vs hand-computed cases	PASS

Oracle coverage	PASS

6.3 RQ2: Selection effectiveness at pilot scale (a null result)

On the verified mutation cohort (nine events, 32 units each), no selector improved on the simple deterministic baselines or on random ordering, the proposed risk_score included; in fact the proposed selector clearly trailed them (Tables 11–15, Figure 4). Diversity-only attains the best mean AUC at 0.886, the cost, history, and change-stage selectors tie at 0.877, random over 20 seeds reaches a median of 0.783 (with a seed range of 0.613–0.854), and the proposed selector is lowest at 0.692. This deficit is not merely noise. Against diversity-only the effect size is small and negative (Cliff's $\delta = -0.30$, Vargha–Delaney Â12 = 0.35), while against the cost baselines and random ordering it is negligible ($|\delta| \leq 0.07$).

The per-event view (Tables 12, 14) explains the aggregate. Eight of the nine mutations are detected, and most are detected at near-zero cost by every selector, so the real differentiator is whether a selector front-loads a cheap detecting unit. The proposed selector reaches the detector marginally earlier on six events, but it loses heavily on the two redundancy mutations (insert_redundant_cx_pair and insert_redundant_x_pair, with per-event AUC of 0.20 and 0.21) and on force_opt_level_1 (0.85): on these events its severity weighting promotes expensive, high-criticality units ahead of the cheap units that actually detect the fault. The ninth mutation, drop_vf2_post_layout, is undetected by every selector, because the black-box quality oracle does not register VF2PostLayout removal on this corpus, which caps every curve at $8/9 \approx 0.889$ (Figure 4).

The null result is not a consequence of uniform cost. The per-unit cost spans 8×10^{-5} s to 0.33 s (Table 6, CV ≈ 1.85); but such heterogeneity is immaterial in absolute terms within a 1.65-second suite, because the detecting units are cheap and plentiful. The eight detectable mutations are caught by many low-cost units, so every ordering reaches a detector within a small fraction of the suite budget, and detection stays essentially flat across the 5–40% budget fractions. Cost-aware prioritization simply cannot help once cheap detectors already saturate early detection.

We therefore report a null result at this scale. Across the nine events the proposed selector shows no advantage over the simple baselines or random ordering; the bootstrap confidence intervals are wide and overlapping (Table 15), and there is only a weak indication that its extra weighting hurts by deferring cheap detectors. In keeping with the claim-scope rule, and because fewer than three forward-regression events are verified (one, H4 in §6.5), we make no comparative-effectiveness or temporal-generalization claim.

All nine mutations share the Qiskit 2.4.2 base revision, so this is not a temporal-generalization test. The configured cutoff (2026-01-01) places all nine mutation split-groups on the test side, no mutation event is used for training, and the selector configurations are frozen (Table 3) and were not tuned on these events. We therefore describe the comparison as a frozen-configuration pilot rather than a held-out evaluation.

Table 11. Frozen-configuration pilot metrics on the mutation cohort (9 events, 32 units each). Lower normalized TTFR and higher AUC are better; overhead is mean selection time.

Selector	r@5%	@10%	@20%	@40%	nTTFR	AUC	overhead (s)
diversity_only	0.89	0.89	0.89	0.89	0.114	0.886	0.0000
cheapest_first	0.89	0.89	0.89	0.89	0.123	0.877	0.0000
history_only	0.89	0.89	0.89	0.89	0.123	0.877	0.0003
change_stage	0.89	0.89	0.89	0.89	0.123	0.877	0.0000

random (median, 20 seeds) †	0.44	0.56	0.72	0.89	0.217	0.783	0.0000
proposed (risk_score)	0.56	0.56	0.67	0.67	0.308	0.692	0.0017

† Random over 20 seeds: per-fraction detection medians 0.44 / 0.56 / 0.72 / 0.89 at 5 / 10 / 20 / 40%; mean AUC median 0.783 (seed range 0.613–0.854); mean norm. TTFR median 0.217 (range 0.146–0.387). The shaded band in Figure 4 shows the 20-seed min–max range.

Table 12. Per-event area under the detection-vs-budget curve (1.0 = detected at ≈ zero cost; 0.0 = never detected). Nine mutation events.

Mutation event	cheap.	divers.	hist.	ch-stage	proposed	rand(med)
disable_optimization_loop	0.97	0.98	0.97	0.97	0.99	0.88
drop_vf2_post_layout	0.00	0.00	0.00	0.00	0.00	0.00
downgrade_routing	0.98	1.00	0.98	0.98	1.00	0.94
insert_redundant_cx_pair	1.00	1.00	1.00	1.00	0.20	0.99
drop_optimization_stage	0.97	1.00	0.97	0.97	0.99	0.91
force_opt_level_1	0.98	1.00	0.98	0.98	0.85	0.85
force_opt_level_0	1.00	1.00	1.00	1.00	1.00	1.00
downgrade_layout_trivial	1.00	1.00	1.00	1.00	1.00	1.00
insert_redundant_x_pair	1.00	1.00	1.00	1.00	0.21	0.71

Table 13. Win/tie/loss of the proposed selector vs each baseline, by per-event normalized TTFR (9 events).

Baseline	win	tie	loss
cheapest_first	5	1	3
diversity_only	3	1	5
history_only	5	1	3
change_stage	5	1	3
random (median)	5	1	3

Table 14. Per-event normalized TTFR by selector (lower is better); reconciles the win/tie/loss in Table 13. Nine mutation events.

Mutation event	cheap.	divers.	hist.	ch-stage	proposed	rand(med)
disable_optimization_loop	0.033	0.017	0.033	0.033	0.009	0.120
drop_vf2_post_layout	1.000	1.000	1.000	1.000	1.000	1.000
downgrade_routing	0.021	0.004	0.021	0.021	0.001	0.063
insert_redundant_cx_pair	0.001	0.001	0.001	0.001	0.804	0.008
drop_optimization_stage	0.027	0.002	0.027	0.027	0.011	0.091
force_opt_level_1	0.016	0.004	0.016	0.016	0.155	0.146
force_opt_level_0	0.001	0.001	0.001	0.001	0.001	0.004
downgrade_layout_trivial	0.002	0.001	0.002	0.002	0.001	0.004
insert_redundant_x_pair	0.002	0.001	0.002	0.002	0.788	0.295

The proposed selector is marginally ahead on the six events whose cheapest detector it reaches marginally earlier, but loses heavily on insert_redundant_cx_pair (nTTFR 0.804) and insert_redundant_x_pair (0.788), where its severity (impact_prior) weighting defers a cheap detecting unit, and on force_opt_level_1 (0.155); drop_vf2_post_layout is an undetected tie. The few large losses pull the mean AUC below every baseline.

Table 15. Effect sizes (Cliff's δ, Vargha–Delaney Â12; proposed vs each baseline on per-event AUC, 9 events) and bootstrap 95% CIs of mean AUC.

Comparison (proposed vs)	Cliff's δ	magnitude	Â12	baseline mean AUC [95% CI]
diversity_only	−0.30	small	0.35	0.886 [0.662, 0.998]
cheapest_first	−0.05	negligible	0.475	0.877 [0.655, 0.993]
history_only	−0.05	negligible	0.475	0.877 [0.655, 0.993]
change_stage	−0.05	negligible	0.475	0.877 [0.655, 0.993]
random (median)	+0.07	negligible	0.537	0.779 [0.750, 0.804]

Proposed selector mean AUC = 0.692 [0.400, 0.912]. Effect sizes are negligible against the cost baselines and random, and small-but-negative against diversity-only; the wide, overlapping CIs at nine events support a claim of no advantage rather than a reliable deficit.

6.4 Ablation and sensitivity of risk_score

To locate the source of the deficit, we ablated each risk_score component over the cached labels, with no re-transpilation (Table 16). Removing the severity term impact_prior raises the mean AUC from 0.692 to 0.716, so this term is actively harmful: a high impact_prior on expensive, high-criticality units defers the cheap units that detect these faults. The novelty term helps only marginally, since the AUC falls to 0.688 without it, while change_stage_match and oracle_confidence are inert on this corpus, because every unit shares the modified stage and the same exact-unitary oracle tier. A sensitivity sweep over the ε floor ({0.01, 0.05, 0.10}) and the exploration reserve ({10, 15, 20}%) leaves the mean AUC unchanged at 0.692, which tells us the result is robust and not a tuning artifact.

Table 16. Ablation of risk_score components (mutation cohort, 9 events; reuses cached labels, no transpilation).

risk_score variant	mean AUC	mean nTTFR	recall@20%
full	0.692	0.308	0.667
− novelty	0.688	0.312	0.667
− change_stage	0.692	0.308	0.667
− oracle_confidence	0.692	0.308	0.667
− impact_prior	0.716	0.284	0.667

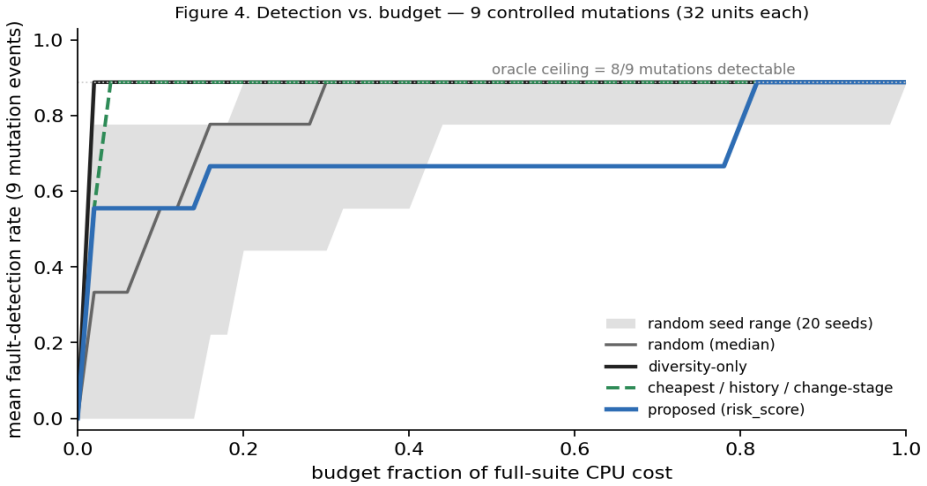


Figure 4 plots the detection-vs-budget curves. Diversity-only and the cost, history, and change-stage baselines reach the 8/9 ≈ 0.889 ceiling almost immediately, random (the median) climbs more gradually, and the proposed selector plateaus lower because it defers cheap detectors on three events. The shaded band shows the 20-seed min–max range for random ordering.

6.5 RQ3: Observability of historical regressions

We attempted five historical events, and we must separate a demonstrated observability gap from cases that are merely unreproduced or under-powered at this scale. Of the five, one (H1) is a built-from-source demonstration that a real regression is invisible to a black-box output oracle, and one (H4) is a verified forward-regression performance event that a multi-run protocol detects (below). The remaining three are inconclusive, and they are not evidence that the oracles under-detect.

- **H1 (ElidePermutations, #14603): demonstrated observability gap.** Both revisions were built from source. The fault belongs to the transpiler_contract_or_metadata channel: it corrupts the virtual_permutation_layout property while leaving the layout-applied output unitary correct. As a result, the output-only semantic oracle reports equivalence on the buggy build, even when the fix's own regression circuit is run through the full preset pass manager. Under the production-pipeline rule the event is therefore rejected as real but production-unobservable. This is the single place where the pilot establishes that a black-box output oracle misses a contract/metadata-channel fault, and Appendix A gives the minimal trigger, the exact SHAs and the corrupted field, the oracle outcome, and the reproduction command. Run retrospectively, a fault-class-matched oracle does see it: fed the same trigger in isolation (the canonical PR-#14603 reproduction), the buggy ElidePermutations raises a KeyError on the unmapped qubit while the fixed pass succeeds, so an isolated-pass/property check detects the divergence that the production output oracle masks (Appendix A). This check is now a first-class Secondary-track oracle in the pipeline: cart historical-run --use-targeted reports contract_metadata_fail on H1 while passing on the fixed anchor, a property-oracle false-positive rate of zero on these events.
- **H3 (VF2Layout non-determinism, #14730): inconclusive (not reproduced).** The determinism-channel fault reproduces only under a noisy device target absent from our coupling-map backend; a strengthened, layout-sensitive determinism oracle observed determinism on our circuit. We cannot conclude anything about oracle sensitivity because the triggering condition was never created. Blocked. This is consistent with a recent dynamic study (a preprint at the time of writing) that executed the Qiskit Terra suite 10,000 times across 23 releases and found genuine flakiness rare but often requiring thousands of runs to detect reliably, well beyond a typical CI budget [36].

- **H4 (VF2PostLayout no-op, #14120): verified (detected).** We re-test it with a multi-run Stage-2 protocol: seven wall-clock timings per build, a pre-declared slowdown threshold, a bootstrap confidence interval, and a Cliff's δ effect size. Run over a width-extended regime of symmetric circuits on a large heavy-hex coupling map at optimization level 3 (the regime issues #14867 and #14120 identify), the candidate's no-op overhead is decisively detected. The median slowdown grows with width from about 1.9× at 16 qubits to more than two orders of magnitude by 27 qubits, with complete sample separation (Cliff's δ = 1.0 and a bootstrap CI well above zero). On the original 8-qubit-capped corpus the single-sample CPU process time stayed below the 20% Stage-1 screen, which is why the pilot first recorded the event as under-powered. H4 is therefore a verified forward-regression performance event, and it validates the primary-CI eligibility rule on a real event; as a single performance-channel event it leaves the null selection result and the three-verified-event temporal-generalization bar unchanged.
- **H2 (#14919) and H5 (#16285): not run.** H2 is expected to share H1's layout-property masking and H5 is an input-specific crash; neither has been reproduced, so neither contributes evidence.

The controlled mutation cohort, by contrast, labels reliably (Table 7). We therefore answer RQ3 narrowly. The pilot demonstrates a single concrete observability gap, namely a contract/metadata-channel regression (H1) that is invisible to black-box output oracles, and it leaves the determinism and small-overhead channels open, since we could not reproduce or adequately power them on commodity hardware. We do not claim that black-box oracles broadly under-detect. What we claim is narrower: at least one important manifestation channel is unobservable to them, and the others remain untested here. Crucially, the gap is closable: a retrospective, fault-class-matched oracle that runs ElidePermutations in isolation does detect H1 (the buggy pass errors on the trigger where the fixed one succeeds), direct evidence for the property/isolated-pass oracle our scale-up proposes. Section 6.6 then quantifies how often such channels occur among real transpiler fixes.

6.6 Observability of real transpiler regressions: a repository-mining study

The H1 result raises a question that the pilot cannot answer on its own: how common are output-invisible regressions in practice? To estimate this, we mined the merged, changelog-listed bug-fix pull requests labelled mod: transpiler in the Qiskit repository and classified each one by the manifestation channel of the fault it fixes (Section 2.2). This follows repository-mining practice for quantum-software bug characterization [37] and the empirical-reporting guidance for quantum-software testing [35]. Two authors independently classified the 26 fixes from their titles, descriptions, and linked issues, using a written codebook and recording a confidence flag for each. Agreement was substantial: Cohen's κ = 0.68 (95% CI [0.38, 0.97], wide at n = 26) on the binary observable-by-output-oracle judgment (84.6% raw agreement) and κ = 0.67 on the seven-class channel (73.1%), and both raters independently identified 10 of the 26 fixes as output-invisible. The four binary disagreements were resolved by discussion; in particular the ElidePermutations case (PR #14603, our H1) was adjudicated as output-invisible on the strength of direct built-from-source evidence. The codebook, both raters' labels, and the confidence flags are released with the artifact. As Table 17 shows, 10 of the 26 fixes (38%, Wilson 95% CI [22%, 57%]; 41% once low-confidence cases are excluded) fall in channels that a black-box output-equivalence oracle cannot observe: corrupted layout or permutation contract metadata (7), non-determinism (2), and a dropped global phase (1, which is invisible to equivalence-modulo-global-phase checks). The built-from-source H1 case is one instance of the largest of these classes; we flag its dual role explicitly, since the event that motivates the taxonomy is also counted in the prevalence, and excluding it leaves 9 of 25 (36%) output-invisible, within the same interval. The observability gap is therefore not an isolated curiosity. A substantial minority of real transpiler regressions are unobservable to output-only oracles, which motivates treating fault-class-matched oracles, such as property and layout comparison and determinism checks, as first-class CI checks. Concretely, adding such oracles would render all 10 of these output-invisible fixes observable; for the largest class we recommend that CI assert the transpiler's layout/permutation contract directly, comparing TranspileLayout (initial_index_layout, final_index_layout), routing_permutation, and the per-pass

virtual_permutation_layout against a trusted baseline, a check our isolated-pass prototype already performs for H1 at negligible cost relative to transpilation.

Table 17. Manifestation channels of 26 mined Qiskit transpiler bug-fixes. “Observable” = detectable by a black-box output-equivalence oracle.

Manifestation channel	Count	Observable?	Example PRs
output_semantic	5	yes	#16428, #16337, #13670
compilation_failure	6	yes	#15626, #16285, #14998
circuit_quality	4	yes	#14667, #13884
performance	1	yes (if over screen)	#14120
contract / metadata	7	no	#14939, #14919, #13945, #13833, #14603
determinism / reproducibility	2	no	#14763, #14730
global phase	1	no	#15943

Threats: classification is title-and-description based, a construct-validity threat that we mitigate by independent double-coding (two authors; Cohen's κ = 0.68, 95% CI [0.38, 0.97] on the binary observability judgment and 0.67 on the channel) and by releasing the codebook, both raters' labels, and the confidence flags for re-coding; the sample is recent merged fixes, not a complete census, so the percentages are indicative rather than population estimates.

7. Discussion

The central lesson of the pilot is methodological rather than a horse-race outcome. Its most concrete empirical finding is an observability gap. A real transpiler regression, H1, lives in the transpiler_contract_or_metadata channel: its layout metadata is incorrect, yet the output unitary it produces is intact, and so a black-box output oracle cannot see it. We are careful not to over-generalize from a single event. The determinism channel (H3) remains open in our data because we could not reproduce it, not because we watched the oracle fail; the small-overhead channel (H4) is, by contrast, now detected once the performance oracle is run as a multi-run protocol over larger symmetric circuits, and is our one verified forward-regression event. The honest reading is therefore a caution rather than a sweeping claim: equivalence checking of compiled outputs is provably insufficient for at least one important class of compiler regression, and the contract/metadata, determinism, and performance channels each plausibly need an oracle of their own. The repository-mining study of §6.6 shows that these output-invisible channels account for roughly 38% of recent real transpiler bug-fixes, which makes the gap quantitatively material rather than anecdotal. The methodology already encodes the design consequences: a controlled mutation cohort serves as a reliable labelling backbone; test provenance is accounted for strictly, since a post-fix test can show that a fault is real (a Secondary claim) but cannot prove that a selector would have caught it in real CI; and fault-class-matched oracles are planned for the scale-up, one of which, an isolated-pass/property check, we already prototype and show to detect H1 retrospectively (§6.5).

The selection result is a null result, and we treat it as informative rather than disappointing. At this scale the transparent risk_score selector did not beat the simple cost, diversity, history, and change-stage baselines, nor random ordering, and it trailed them because its severity weighting can defer a cheap detecting test (§6.3). An ablation isolates the cause: once the severity term (impact_prior) is removed, the AUC returns to baseline level (Table 16), so the design is not wrong in principle but is mis-calibrated for a regime in which detection cost and fault severity are decoupled. More broadly, no method separates here, for the reason established in §6.3: cheap detectors saturate detection so early that ordering cannot matter. Whether the extra structure in risk_score helps is therefore still an open question, one that only a larger, cost-heterogeneous corpus with verified forward regressions can settle, and the claim-scope rule correctly forbids any comparative-effectiveness claim from these nine

events. The most natural next prototype simply drops the impact_prior term, which on its own recovers baseline-level AUC (Table 16); we offer this as a lesson rather than a result, since it is a post-hoc adjustment evaluated on the very same nine events.

8. Threats to Validity

Construct. The largest threat is oracle observability (§7). A black-box oracle can miss contract/metadata-channel faults, as we demonstrate for H1, and it may also miss determinism- and performance-channel faults, which we did not test here, so a “pass” does not certify that a circuit is fault-free. We mitigate this by separating the cohorts, by using an empirical, confirmation-based false-positive rate, and by recording which oracle tier produced each label and which manifestation channel each historical fault occupies. The provenance rule also prevents post-fix tests from inflating the Primary claim.

Internal. Timing on a single laptop is noisy, so we calibrate cost with CPU process time, treat performance as exploratory, and exclude high-load runs. Write-once raw artifacts, fixed seeds, and labels regenerated from the raw evidence all support reproducibility. Test non-determinism is a known confound in regression testing [38]. We therefore treat determinism and reproducibility as their own manifestation channel rather than folding them into correctness, and, consistent with the evidence that flaky tests are most detectable when newly added [39], we confine post-fix targeted triggers to the Secondary evaluation. A further internal-validity threat is that the temporal split is, at present, vacuous: all nine mutations share the Qiskit 2.4.2 base, so every split-group lands on the test side and nothing trains, and the anti-leakage machinery is thus demonstrated where there is nothing to leak; a genuine temporal evaluation awaits the forward-regression events of the scale-up.

External. The corpus is small and specific to the Qiskit 2.x transpiler, so the results are pilot-scoped and must not be generalized without the planned scale-up. The verified events are nine quality mutations plus one performance forward-regression (H4), which may not represent the full fault distribution. There is also a scope limitation in the evaluation itself: the primary results exercise only the exact-unitary and quality-oracle tiers, whereas the sampled-statevector and noisy-target determinism oracle tiers are described in the methodology but deferred; the property/layout oracle and the robust multi-run performance oracle are now implemented, detecting H1 and verifying H4 respectively. The evaluated system is therefore narrower than the oracle architecture we discuss, and closing that gap is a central part of the scale-up. The historical-event reproduction is likewise only partial: H2, H3, and H5 were not run and are not reproducible as submitted, so the verified historical evidence reduces to H4 (a forward performance regression) and the H1 observability demonstration.

Conclusion. With so few events, statistical comparison is under-powered. We therefore avoid significance claims, report per-event outcomes alongside the averages, and bind comparative claims with the rule that requires at least three forward-regression events. The leakage-absence gate guards against optimistic bias. The null selection result is itself a per-event finding, not a powered comparison.

9. Limitations and Future Work

To summarize, the study is a feasibility pilot: one forward-regression event (H4) is verified through a multi-run performance protocol while the remaining verified cohort is mutation-only, and the selector comparison yields a null result. At this scale the proposed selector showed no advantage over the simple baselines or random ordering, and, as established in §6.3, cheap detectors saturate detection early, leaving no regime in which cost-aware ordering can separate the methods. The deferred scale-up will do four things. First, it will expand the corpus toward at least three verified forward-regression events, and will deliberately include events whose detecting tests are expensive or sparse, so that cost-aware ordering finally has a regime in which to separate methods; the present corpus fails to discriminate because cheap detectors dominate, not because the costs are uniform. Second, it will add fault-class-matched oracles: a property and layout oracle comparing TranspileLayout and routing_permutation, a noisy-target determinism oracle, and three-run-median performance on

controlled hardware, all in a clearly labelled retrospective track. Third, it will introduce an optional comparison against logistic-regression and gradient-boosted-tree selectors under identical anti-leakage rules. Fourth, it will trace selected fix-boundary events back to their true introducing commits through bisection, converting them into the forward cohort. Only once at least three forward-regression events are verified do comparative-effectiveness claims about temporal generalization become admissible.

10. Conclusion

In this paper we asked whether regression-test selection and prioritization, long established for classical software, can be realized for the quantum transpiler in a leakage-safe and cost-aware manner, and what such an approach achieves at feasibility scale. We designed a methodology that separates change events into non-pooled cohorts, applies tiered layout-aware oracles with an empirical false-positive rate, scores tests with a transparent selector that distinguishes fault severity from detection confidence, and is guarded by six implementation-validity gates and a strict claim-scope rule. We then realized the methodology in an open prototype and evaluated it on a Qiskit 2.x micro corpus.

We report our empirical findings without embellishment. Across nine controlled mutations, the transparent selector did not outperform the simple cost, diversity, history, and change-stage baselines, nor random ordering. A per-unit cost analysis (§6.3) explains why, and an ablation pinpoints the selector's severity term as the component responsible, which rules out a mere tuning issue. Separately, our attempt to reproduce five real historical regressions from source produced one clear positive finding, namely an ElidePermutations regression whose corrupted internal layout metadata is invisible to output-equivalence oracles. The remaining cases could not be reproduced or adequately powered on commodity hardware, and we report them as inconclusive rather than as evidence of oracle weakness.

Taken together, the study contributes a rigorous and reproducible evaluation methodology, an honest feasibility baseline against which future selectors can be compared, and a concrete demonstration that equivalence checking of compiled outputs is insufficient to observe an important class of transpiler-contract faults. None of these outcomes amounts to a comparative-effectiveness claim. What they do provide is a credible foundation, together with a pre-specified path toward a larger empirical evaluation that uses verified forward regressions, cost-heterogeneous corpora, and fault-class-matched oracles.

11. Artifact Availability

To support review and reproduction, the artifact is archived at https://doi.org/10.5281/zenodo.21020113 and the public repository is https://github.com/furqan-nr/quantum-transpiler-regression-testing, containing: the prototype (cart) and CLI; the automated test suite and six validity gates; the static manifest and mutation engine; the event ledger with exact candidate/baseline commit SHAs; the frozen change–stage map, pre-declared thresholds, RNG seeds, and the frozen risk_score configuration (Table 3); a pinned environment lock (requirements.lock); the write-once raw oracle artifacts and derived labels; the generated evaluation.json; the repository-mining dataset (data/observability_mining.csv); and an OSI-approved open-source license.

The frozen-configuration pilot of Section 6 is reproduced exactly by two commands:

```
python -m cart.cli ground-truth --unit-limit 64
python -m cart.cli evaluate --split all --random-seeds 20
```

These write Tables 11–14, the per-fraction random aggregates, the cost-distribution summary, and the detection-vs-budget data of Figure 4 to a single evaluation.json. Building the historical events (e.g.,

H1) additionally requires a Rust toolchain to compile the per-event Qiskit revisions from source; the per-event recipe and reproduction command are given in Appendix A.

Statements and Declarations

Funding. The authors received no specific grant from any funding agency, commercial, or not-for-profit sector for this work.

Competing Interests. The authors declare no competing financial or non-financial interests.

Data Availability. All data supporting the results, namely the change-event ledger, the write-once raw oracle artifacts, the derived labels, and the generated evaluation reports, are available in the public repository (https://github.com/furqan-nr/quantum-transpiler-regression-testing) and archived at https://doi.org/10.5281/zenodo.21020113; see Section 11.

Code Availability. The cart prototype, its command-line interface, and the build scripts are available in the public repository (https://github.com/furqan-nr/quantum-transpiler-regression-testing, archived at https://doi.org/10.5281/zenodo.21020113) under the MIT open-source license; see Section 11.

Author Contributions. F. Nasir conceived the study, implemented the prototype, conducted the evaluation, and wrote the manuscript. A. Shah supervised the work and contributed to the study design and the critical revision of the manuscript. I. Alam contributed to the methodology and the analysis and reviewed the manuscript. All authors read and approved the final manuscript.

Ethics Approval and Consent. Not applicable; the study involves no human or animal subjects.

Use of AI Tools. AI-based assistants were used to support manuscript drafting and software implementation under the authors' supervision; the authors verified all results and take full responsibility for the content. Specifically, AI assistance covered prose drafting and editing, portions of the prototype code and test scaffolding, and results formatting; the study design, labelling, empirical findings, and all numerical results were produced and verified by the authors.

Appendix A. Direct evidence for H1 (ElidePermutations, PR #14603)

H1 is the pilot’s one demonstrated observability gap, so we record its evidence explicitly.

- **Revisions (built from source).** Candidate (buggy) 7c3890da097c851876b86453a1da6ee7d3208048 → baseline (fixed) 96fda18888de4492f37ccdb93faf15dc014c99eb; fix-boundary differential, PR #14603, fix-commit dated 2025-06-16.
- **Minimal trigger.** A circuit containing a PermutationGate together with two-qubit gates (the unit trig-h1-elide-permutations, drawn from the fix’s own regression test), transpiled through the full preset pass manager at optimization level 3 on backends none and line.
- **Corrupted field.** The buggy ElidePermutations pass corrupts the virtual_permutation_layout property recorded in the TranspileLayout / output metadata; the layout-applied output unitary is left correct.
- **Oracle outcome.** The layout-normalized exact-unitary oracle reports EQUIVALENT (pass) on the buggy candidate, identical to the baseline result, on both backends. The output-only oracle therefore cannot distinguish buggy from fixed; the fault is only visible by comparing the recorded permutation property directly (a contract/metadata check the production pipeline does not run). Per the production-pipeline rule the event is rejected, not credited. As a retrospective, fault-class-matched check we additionally run ElidePermutations in isolation on the same trigger (the canonical PR-#14603 reproduction): the buggy build raises a KeyError on the unmapped qubit while the fixed build returns a valid circuit, so an isolated-pass/property oracle detects the divergence the production output oracle masks. This deviation from the production pipeline is reported only in the Secondary retrospective track and is reproduced by

python scripts/verify_h1_isolated.py; the same check is wired into the runner as a contract oracle, so python -m cart.cli historical-run --event hist-elide-permutations-mapping --use-targeted reports contract_metadata_fail on the buggy build and pass on the fixed anchor.

- **Reproduction.** Build both revisions with the per-event from-source runner, then run:

```
python -m cart.cli historical-run --event hist-elide-permutations-mapping --use-targeted
```

The recorded per-run property values (baseline vs candidate virtual_permutation_layout) are written to the event's write-once raw artifact and are included in the deposited archive (Section 11).

References

[1] Paltenghi M, Pradel M (2022) Bugs in quantum computing platforms: an empirical study. Proc ACM Program Lang (OOPSLA). arXiv:2110.14560

[2] Nation PD et al (2024) Benchmarking the performance of quantum computing software for quantum circuit creation, manipulation and compilation. Nat Comput Sci

[3] Yoo S, Harman M (2012) Regression testing minimization, selection and prioritisation: a survey. Softw Test Verif Reliab 22(2):67–120

[4] Machalica M, Samylkin A, Porth M, Chandra S (2019) Predictive test selection. In: Proc ICSE-SEIP. arXiv:1810.05286

[5] Paltenghi M, Pradel M (2023) MorphQ: metamorphic testing of the Qiskit quantum computing platform. In: Proc ICSE. doi:10.1109/ICSE48619.2023.00202

[6] Kitt L et al (2024) MorphQ++: a reproducibility study of metamorphic testing on quantum compilers. In: Proc IEEE/ACM ASE Workshops. doi:10.1145/3695750.3695823

[7] Pontolillo G et al (2025) QuCheck: a property-based testing framework for quantum programs in Qiskit. arXiv:2503.22641

[8] Luo J et al (2026) QEMI: a quantum software stacks testing framework via equivalence modulo inputs. arXiv preprint

[9] Javadi-Abhari A, Treinish M, Krsulich K et al (2024) Quantum computing with Qiskit. arXiv:2405.08810

[10] Paltenghi M, Pradel M (2024a) A survey on testing and analysis of quantum software. arXiv:2410.00650

[11] Barr ET, Harman M, McMinn P, Shahbaz M, Yoo S (2015) The oracle problem in software testing: a survey. IEEE Trans Softw Eng 41(5):507–525

[12] Burgholzer L, Wille R (2021) Advanced equivalence checking for quantum circuits. IEEE Trans Comput-Aided Des Integr Circuits Syst 40(9):1810–1824

[13] Elbaum S, Malishevsky AG, Rothermel G (2002) Test case prioritization: a family of empirical studies. IEEE Trans Softw Eng 28(2):159–182

[14] Elbaum S, Malishevsky A, Rothermel G (2001) Incorporating varying test costs and fault severities into test case prioritization. In: Proc ICSE

[15] Rothermel G, Untch RH, Chu C, Harrold MJ (2001) Prioritizing test cases for regression testing. IEEE Trans Softw Eng 27(10):929–948

[16] Elbaum S, Rothermel G, Penix J (2014) Techniques for improving regression testing in continuous integration development environments. In: Proc ACM SIGSOFT FSE, pp 235–245

[17] Elsner D, Hauer F, Pretschner A, Reimer S (2021) Empirically evaluating readily available information for regression test optimization in continuous integration. In: Proc ACM SIGSOFT ISSTA, pp 491–504

[18] Peng Q, Shi A, Zhang L (2020) Empirically revisiting and enhancing IR-based test-case prioritization. In: Proc ACM SIGSOFT ISSTA, pp 324–336

[19] Spieker H, Gotlieb A, Marijan D, Mossige M (2017) Reinforcement learning for automatic test case prioritization and selection in continuous integration. In: Proc ACM SIGSOFT ISSTA, pp 12–22

[20] Prado Lima JA, Vergilio SR (2022) A multi-armed bandit approach for test case prioritization in continuous integration environments. IEEE Trans Softw Eng 48(2):453–465

[21] Pan R, Bagherzadeh M, Ghaleb TA, Briand L (2022) Test case selection and prioritization using machine learning: a systematic literature review. Empir Softw Eng 27(2):29

[22] Aparicio-Morales A, Moguel E, Garcia-Alonso J, Murillo JM (2024) An overview of quantum software engineering in Latin America. Quantum Inf Process 23

[23] Huang Y et al (2024) Qubit mapping: the adaptive divide-and-conquer approach. Quantum Inf Process 23

[24] Wang J, Zhang Q, Xu GH, Kim M (2021) QDiff: differential testing of quantum software stacks. In: Proc IEEE/ACM ASE, pp 692–704

[25] Ye J et al (2026) QSPE: enumerating skeletal quantum programs for quantum library testing. arXiv preprint

[26] Choudhury N et al (2025) Concolic testing for quantum compilers. In: Proc IEEE ICCD

[27] Tao R, Shi Y, Yao J, Li X, Javadi-Abhari A, Cross AW, Chong FT, Gu R (2022) Giallar: push-button verification for the Qiskit quantum compiler. In: Proc ACM SIGPLAN PLDI, pp 641–656

[28] Paltenghi M, Pradel M (2024b) Analyzing quantum programs with LintQ: a static analysis framework for Qiskit. Proc ACM Softw Eng 1(FSE):1135–1157

[29] Zhao P, Zhao J, Miao Z, Lan S (2023) Bugs4Q: a benchmark of existing bugs to enable controlled testing and debugging studies for quantum programs. J Syst Softw 205:111805

[30] Mendiluze E, Ali S, Yue T, Arcaini P (2021) Muskit: a mutation analysis tool for quantum software testing. In: Proc IEEE/ACM ASE

[31] Fortunato D, Campos J, Abreu R (2022) Mutation testing of quantum programs: a case study with Qiskit. IEEE Trans Quantum Eng 3:1–17

[32] Ramalho NCL et al (2024) Testing and debugging quantum programs: the road to 2030. ACM Trans Softw Eng Methodol

[33] Wojciechowski K et al (2026) Open QBench: a benchmarking framework for evaluating quantum computing platforms. Quantum Inf Process

[34] Mendiluze Usandizaga E, Yue T, Arcaini P, Ali S (2023) Quantum circuit mutants: empirical analysis and recommendations. Empir Softw Eng 28:6

[35] Li Y et al (2026) A methodological analysis of empirical studies in quantum software testing. arXiv preprint

[36] Kim D et al (2025) Detecting flakiness in quantum software: a dynamic testing approach. Preprint (venue/DOI to be confirmed)

[37] Yousuf MM et al. (2025) Characterizing bugs and quality attributes in quantum software: a large-scale empirical study. arXiv preprint.

[38] Luo Q, Hariri F, Eloussi L, Marinov D (2014) An empirical analysis of flaky tests. In: Proc ACM SIGSOFT FSE, pp 643–653

[39] Lam W, Winter S, Wei A, Xie T, Marinov D, Bell J (2020) A large-scale longitudinal study of flaky tests. Proc ACM Program Lang 4(OOPSLA):1–29